

COMSATS UNIVERSITY ISLAMABAD

LAHORE CAMPUS

MACHINE LEARNING

LAB TASKS

NAME	REG NO
ZEESHAN AHMAD ABBASI	FA24-BSE-181

ACADEMIC YEAR: 2024 – 2028

BS SOFTWARE ENGINEERING

SUBMITTED TO:

DR. RAO ADEEL NAWAB

DATE OF SUBMISSION:

11/10/2026

Abstract

This report describes the design, implementation and evaluation of a Heart Disease Prediction System developed for Assignment 2 of Machine Learning Fundamentals (AIC354). Following the instructor's Titanic survival workflow, the project selects exactly four input attributes `Age`, `Sex`, `ChestPainType` and `MaxHR` and trains a Support Vector Classifier (SVC) to predict whether heart disease is present.

The final model is trained on the Kaggle Heart Failure Prediction dataset (918 records, 12 columns). Numeric inputs are standardised and categorical inputs are onehot encoded inside a single scikitlearn pipeline, which is saved as one file and loaded by a Streamlit web application. On a heldout test set of 184 records the model reaches accuracy 0.788, precision 0.794, recall 0.833, F1score 0.813 and ROCAUC 0.859. The original UCI Cleveland experiment is retained as a baseline (accuracy 0.600 on 60 test records); because the datasets and test sets differ, this is not a controlled comparison.

The application runs locally. A previously deployed Streamlit Community Cloud app exists, but it has not been verified to contain the updated model, so cloud deployment of the final version is reported as pending. The system is an educational demonstration and has not been clinically validated.

Table of Contents

Chapter 1 Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Motivation 1
1.4 Objectives 1
1.5 Scope 1
1.6 Limitations 2
Chapter 2 Dataset Description 3
2.1 Primary Dataset: Kaggle Heart Failure Prediction 3
2.2 Selected Input Features 3
2.3 Target Variable 4
2.4 Data Quality and Preparation 4
2.5 Baseline Dataset: UCI Cleveland 4
Chapter 3 Methodology 5
3.1 Dataset Acquisition 5
3.2 Data Inspection 5
3.3 Feature Selection 5
3.4 Train/Test Splitting 5
3.5 Preprocessing 5
3.6 Model Training 5
3.7 Evaluation 6
3.8 Model Serialisation 6
3.9 Streamlit Application Integration 6
3.10 Cloud Deployment 6
Chapter 4 System Design and Implementation 7
4.1 Project Files 7
4.2 System Architecture 7
4.3 User Interface 7
4.4 Why the Same Pipeline Is Used for Training and Prediction 8
Chapter 5 Model Evaluation and Results 9
5.1 Kaggle Model (Primary) 9
5.2 Cleveland Baseline 9

5.3 Comparison and Interpretation 10
5.4 Limitations of the Results 10
Chapter 6 Streamlit Application and Deployment 11
6.1 Purpose and Operation 11
6.2 Running Locally 11
6.3 Deployment to Streamlit Community Cloud 11
6.4 Deployment Status 11
Chapter 7 Testing 12
Chapter 8 Limitations and Future Improvements 13
8.1 Limitations 13
8.2 Possible Future Improvements 13
Chapter 9 Conclusion 14
References 15

Abbreviations

Abbreviation	Meaning
AUC	Area Under the (ROC) Curve
EDA	Exploratory Data Analysis
F1	Harmonic mean of precision and recall
FN / FP / TN / TP	False negative / false positive / true negative / true positive
ML	Machine Learning
RBF	Radial Basis Function (the default SVC kernel)
ROC	Receiver Operating Characteristic
SVC / SVM	Support Vector Classifier / Support Vector Machine
UCI	University of California, Irvine (Machine Learning Repository)
UI	User Interface

Chapter 1 Introduction

1.1 Background

Heart disease is a group of conditions affecting the heart and blood vessels. Clinicians assess risk using patient history, symptoms and test results. Public datasets that record such measurements together with a diagnosis allow machine learning models to be trained and studied as classification problems, where each patient record is assigned to one of two classes: heart disease present or absent.

Machine learning is useful here because a model can learn patterns across many records and then score a new record consistently. In this project that idea is used only for learning and demonstration; it is not presented as a clinical decision tool.

1.2 Problem Statement

Given four attributes of a person age, sex, chest pain type and maximum heart rate achieved predict whether the record belongs to the heartdisease class. The assignment constrains the model to exactly four input attributes from the heartdisease data.

1.3 Motivation

The assignment asks students to build a complete machinelearning system, from data collection to cloud deployment, by adapting the instructor's Titanic Passenger Survival Prediction code [3] to a new problem. Building the full path data, preprocessing, training, evaluation, saving, a web interface and deployment shows how a model becomes something a user can interact with.

1.4 Objectives

- Obtain and inspect a heartdisease dataset and document its quality.
- Select exactly four input attributes and define a binary target.
- Train and evaluate a Support Vector Classifier using a reproducible, leakagefree pipeline.
- Save the fitted model and reload it to make predictions without retraining.
- Provide a Streamlit web application with the four inputs and prepare it for cloud deployment.
- Compare the new Kagglebased experiment with the original Cleveland baseline, stating the limits of that comparison.

1.5 Scope

The project covers a single binaryclassification model, a training script, a Jupyter notebook, a Streamlit application and project documentation. The original assignment is called the UCI Heart Disease dataset [1]. The project began with its Cleveland subset and was later upgraded to the larger Kaggle dataset [2], which is a compilation that includes UCIsourced records; the Cleveland experiment is kept as the baseline.

1.6 Limitations

- Only four predictors are used, so many clinically relevant variables are ignored.
- The data are public, historical and polled from several sources; they may not represent any population.
- Evaluation uses one fixed train/test split; no external validation was performed.
- The system outputs a class label only. It is not a diagnostic tool and does not replace medical professionals.

Chapter 2 Dataset Description

2.1 Primary Dataset: Kaggle Heart Failure Prediction

The final model uses `data/kaggle_heart_failure/heart.csv`, downloaded from the Kaggle dataset Heart Failure Prediction Dataset published by fedesoriano [2]. Kaggle lists the licence as Open Database License (ODbL 1.0). The dataset description states that it was created by combining five heartdisease datasets (including Cleveland), so records from the Cleveland source are also present in it.

Table 1. Summary of the Kaggle dataset (verified from the CSV file)

Property	Value
Source URL	https://www.kaggle.com/datasets/fedesoriano/heartfailureprediction
File used	heart.csv
Records × columns	918 × 12
Columns	Age, Sex, ChestPainType, RestingBP, Cholesterol, FastingBS, RestingECG, MaxHR, ExerciseAngina, Oldpeak, ST_Slope, HeartDisease
Target	`HeartDisease` (0 = no heart disease, 1 = heart disease)
Class counts	0: 410 | 1: 508
Missing values	0
Fully duplicated records	0
Licence (as listed on Kaggle)	ODbL 1.0

2.2 Selected Input Features

Exactly four predictors are used. Their names were checked against both the CSV header and the code in `heart_pipeline.py` (`FEATURES = ["Age", "Sex", "ChestPainType", "MaxHR"]`).

Table 2. The four selected features

Feature	Meaning	Type	Observed values
`Age`	Age in years	Numeric (integer)	28–77 (mean 53.5)
`Sex`	Sex of the patient	Categorical (nominal)	M (725), F (193)
`ChestPainType`	Chestpain type: TA typical angina, ATA atypical angina, NAP nonanginal pain, ASY asymptomatic	Categorical (nominal)	ASY (496), NAP (203), ATA (173), TA (46)
`MaxHR`	Maximum heart rate achieved (beats per minute)	Numeric (integer)	60–202 (mean 136.8)

The reason for choosing these four is that they mix demographic variables (age, sex) with a symptom (chestpain type) and an exercisetest measurement (maximum heart rate), all of which are complete in the data and are commonly used in heartdisease examples. This is a teaching choice, not a clinical recommendation.

2.3 Target Variable

`HeartDisease` takes the values 0 and 1 only. In the dataset documentation, 1 means heart disease and 0 means normal. It is kept out of the feature matrix; the code asserts that the matrix has four columns and does not contain the target.

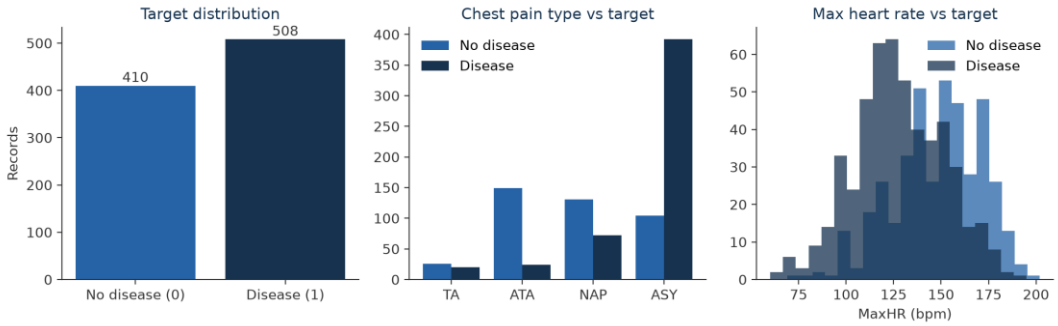


Figure 1. Class distribution, chestpain type and maximum heart rate by target class in the Kaggle dataset (all 918 records).

2.4 Data Quality and Preparation

- **Missing values:** none (0). Nothing was imputed and no value was silently replaced by zero.
- **Duplicates:** 0 fully duplicated records, so none were removed. 59 records share their four predictor values with an earlier record; this is expected because the features take limited distinct values, and deleting them would remove genuine patients.
- **Zero values:** `Cholesterol` equals 0 in 172 records and `RestingBP` equals 0 in 1 record. Such values are physiologically implausible and appear to represent missing data. Neither column is used by the model, so those records were kept; the columns would need handling if they were ever added as inputs.
- **Validity checks:** the cleaning function keeps only records with valid `Sex` and `ChestPainType` categories, plausible `Age` and `MaxHR` ranges and a 0/1 target. It works on a copy; the raw file is not modified. No record was dropped.

2.5 Baseline Dataset: UCI Cleveland

The original project used the Cleveland subset of the UCI Heart Disease dataset [1] (`processed.cleveland.data`, 303 records, 14 columns). 6 records contain missing values (marked `?`) and were dropped, leaving 297. Its target `num` ranges from 0 to 4; the baseline converts it to binary using `num > 0`. The four corresponding features are `age`, `sex` (1/0), `cp` (integer codes 1–4) and `thalach`.

Primary versus baseline. The Kaggle dataset is the primary dataset for the final model; the Cleveland dataset is used only for the baseline experiment. The two use different column names, category encodings and target definitions, and they overlap in source records, so they were never concatenated.

Chapter 3 Methodology

The workflow follows the stages of the instructor's Titanic example and was implemented in the notebook and in `train_model.py`, which share the code in `heart_pipeline.py`.

3.1 Dataset Acquisition

The Kaggle archive was downloaded with the Kaggle commandline tool and extracted to `data/kaggle_heart_failure/heart.csv`. The original Cleveland file was left untouched.

3.2 Data Inspection

The notebook displays the first rows, the shape, column names and data types, and checks missing values, duplicates, class balance and the value ranges of the four features (Chapter 2).

3.3 Feature Selection

The assignment requires exactly four inputs, so the model is trained only on `Age`, `Sex`, `ChestPainType` and `MaxHR`. Limiting the inputs also keeps the web form short and the pipeline easy to explain.

3.4 Train/Test Splitting

The cleaned data (918 records) are split 80/20 with `train_test_split (test_size=0.2, random_state=0, stratify=y)`. This gives 734 training and 184 test records. Stratification keeps the class proportions similar in both parts. The test set is used only for the final evaluation.

3.5 Preprocessing

Preprocessing is defined once, as a `ColumnTransformer` inside a scikitlearn `Pipeline` [5]–[7]:

- **Numeric features (Age, MaxHR):** `StandardScaler` rescales each to zero mean and unit variance. This matters for an RBFkernel SVC because the kernel depends on distances between points.
- **Categorical features (Sex, ChestPainType):** `OneHotEncoder(handle_unknown='ignore')` creates one 0/1 column per category. Because the categories are nominal, integer codes would wrongly imply an order. The category lists are set explicitly, and an unseen category produces an allzero encoding instead of an error.

Because the transformers are part of the pipeline, their parameters (means, standard deviations, categories) are learned from the training split only and then applied unchanged to test and user data. This prevents testset information leaking into training.

3.6 Model Training

The classifier is `SVC (gamma='auto', random_state=0)` [4], the same algorithm as the Titanic example and the original Cleveland experiment. A Support Vector Classifier finds a boundary that separates the two classes with the largest margin; the default RBF kernel lets that boundary be nonlinear. The pipeline is fitted on the training split.

As an optional check, the notebook compares SVC with Logistic Regression and Random Forest on the same features and split, using 5fold crossvalidation inside the training data. SVC remained the default model; the crossvalidated differences between SVC and Logistic Regression were small compared with their variation. This comparison is described in the notebook.

3.7 Evaluation

The fitted pipeline predicts the heldout test records. Accuracy, precision, recall, F1score and the confusion matrix are computed from those predictions. ROCAUC is computed from the SVC decision scores, which rank cases but are not calibrated probabilities; the application therefore does not display probabilities.

3.8 Model Serialisation

The complete fitted pipeline is saved with `joblib` in `heart_disease_model.pkl`, together with metadata: the ordered feature names, the target definition, the category lists, the scikitlearn version and the test metrics. `train_model.py` writes to a temporary file, reloads it, checks that the reloaded model reproduces the metrics, and only then replaces the model file.

3.9 Streamlit Application Integration

`app.py` loads the saved pipeline once at startup, builds a onerow table from the four inputs in the stored feature order and calls `predict`. The application needs no dataset at running time.

3.10 Cloud Deployment

The application is intended for Streamlit Community Cloud [9]. Deployment requires pushing the repository to GitHub, redeploying the app, and choosing a Python version compatible with the pinned packages in `requirements.txt`. As of the verification date, this step has not been confirmed (Chapter 6).

Chapter 4 System Design and Implementation

4.1 Project Files

File	Role
`Heart_Disease_Prediction.ipynb`	Assignment notebook: loads both datasets, inspects and cleans the Kaggle data, trains and evaluates the model, compares with the Cleveland baseline, saves and reloads the model.
`heart_pipeline.py`	Shared code: column definitions, loading and cleaning, split, pipeline construction, metric calculation and the Cleveland baseline run.
`train_model.py`	Commandline training script. Validates the CSV, trains, evaluates, saves the artifact and runs a reload check.
`app.py`	Streamlit application with the four inputs; loads the model and displays the prediction and a disclaimer.
`heart_disease_model.pkl`	Final saved artifact: fitted pipeline (preprocessing + SVC) and metadata, trained on the Kaggle data.
`baseline_cleveland_model.pkl`	The earlier SVC trained on the four Cleveland features, kept for reference. It is a plain SVC without a preprocessing pipeline.
`requirements.txt`	Pinned dependencies (streamlit, scikitlearn, pandas, numpy, joblib).
`processed.cleveland.data`, `data/kaggle_heart_failure/heart.csv`	Baseline and primary datasets.

4.2 System Architecture

Figure 3 shows how a prediction is produced. The four values are entered in the browser, assembled into a onerow table whose columns follow the saved feature order, and passed to the loaded pipeline. The pipeline scales and encodes them exactly as during training and the SVC returns class 0 or 1, which the page displays as text.

4.3 User Interface

The interface (taken from `app.py`) has exactly four inputs:

Label	Widget	Options / range	Value passed to the model
Age (years)	Number input	28–77, default 54 (the range of the training data)	Age (integer)
Sex	Select box	Male, Female	M or F
Chest Pain Type	Select box	Typical Angina (TA), Atypical Angina (ATA), NonAnginal Pain (NAP), Asymptomatic (ASY); default ASY	TA, ATA, NAP or ASY
Max Heart Rate Achieved	Number input	60–202, default 140	MaxHR (integer)

Pressing **Predict** shows a red “Heart Disease Indicated” message for class 1 or a green “No Heart Disease Indicated” message for class 0, and an expandable view of the exact input sent to the model. A caption states that the tool is educational, not a diagnostic device and not clinically validated. If the model file is missing or cannot be loaded (for example after a scikitlearn version mismatch) the app shows an explanatory error instead of failing silently.

4.4 Why the Same Pipeline Is Used for Training and Prediction

A model only works on inputs prepared in the same way as its training data. If the app encoded `ChestPainType` differently, or forgot to scale `Age`, the numbers reaching the SVC would have a different meaning and predictions would be wrong without any error message. Saving preprocessing and classifier together as one object removes that risk: the app passes raw user values and the pipeline performs every transformation.

Chapter 5 Model Evaluation and Results

All values in this chapter were recomputed during report preparation by loading the saved model and the datasets and applying the same split. The Kaggle values match those stored inside the model artifact and in `metrics.json` (artifact match: True). The Cleveland baseline was recomputed with the original configuration and gives the same 60.00 % accuracy printed in the original notebook.

5.1 Kaggle Model (Primary)

Metric	Value	Meaning
Accuracy	0.788	Share of all test records classified correctly.
Precision	0.794	Of the records predicted as disease, the share that truly had disease.
Recall	0.833	Of the records that truly had disease, the share the model found.
F1score	0.813	Harmonic mean of precision and recall.
ROCAUC	0.859	Ability to rank diseased records above healthy ones (from SVC decision scores).

The confusion matrix contains 60 true negatives, 22 false positives, 17 false negatives and 85 true positives. The test set holds 82 records without and 102 records with heart disease. The model finds most diseased records (recall above 0.8) but also raises false alarms for about a quarter of healthy records; in a medical setting a missed case is generally more serious than a false alarm, so recall deserves attention, but this model is not suitable for such use.

5.2 Cleveland Baseline

Metric	Value
Accuracy	0.600
Precision	0.611
Recall	0.393
F1score	0.478
ROCAUC	0.604

The baseline is an `SVC(gamma='auto', random_state=0)` trained on the raw integer features `age`, `sex`, `cp`, `thalach` of 237 Cleveland records and tested on 60. Its confusion matrix has 25 true negatives, 7 false positives, 17 false negatives and 11 true positives. The saved `baseline_cleveland_model.pkl` gives 0.600 accuracy on the same split, consistent with this table.

5.3 Comparison and Interpretation

	Cleveland baseline	Kaggle model
Records used	297	918
Train / test	237 / 60	734 / 184
Feature encoding	Raw integer codes, unscaled	Scaled numerics, onehot categories
Target	`num > 0`	`HeartDisease`
Accuracy	0.600	0.788
Precision	0.611	0.794
Recall	0.393	0.833
F1score	0.478	0.813
ROCAUC	0.604	0.859

With only 60 test records the baseline metrics are very noisy: a single prediction changes accuracy by about 1.7 percentage points. The Kaggle test set is larger (184), but it is still one split and no confidence intervals were computed. One more difference deserves mention: the Kaggle pipeline scales its numeric inputs, whereas the baseline does not, so preprocessing is a second reason the numbers differ.

5.4 Limitations of the Results

- Only four features are used, which caps the information the model can use.
- Results come from a single stratified split; other seeds would give somewhat different numbers.
- The Kaggle data are maledominated (725 male versus 193 female records), so performance may differ across sexes and populations.
- No external dataset was used to check generalisation, and no calibration of probabilities was done.

Chapter 6 Streamlit Application and Deployment

6.1 Purpose and Operation

The application lets a user enter the four attributes and see the model's predicted class. The prediction is computed dynamically from the saved pipeline for each click; nothing is hardcoded or random. The saved file is read from the same folder as `app.py` using a relative path, and it is cached so it is loaded only once.

6.2 Running Locally

From the project folder:

```
pip install r requirements.txt
python train_model.py   #
streamlit run app.py
```

6.3 Deployment to Streamlit Community Cloud

Deployment steps are: (1) push the repository to GitHub; (2) in Streamlit Community Cloud, redeploy the existing app or create a new one that points to `app.py`; (3) in the advanced settings choose Python 3.12 or newer, to match the pinned package versions; (4) open the deployed app, enter test values and confirm that a prediction appears. If package versions are changed, the model should be retrained so that the saved file matches the installed scikitlearn.

6.4 Deployment Status

Item	Status
Previously recorded URL	https://heartdiseasepredictiongg.streamlit.app/
Check on 11 Oct 2026	The page loads and shows “This app has gone to sleep due to inactivity”. The app itself was not woken.
Model version running there	Not verified. It should be treated as the earlier (Clevelandbased) version.
Local repository	The local `main` branch matched the remotetracking branch `origin/main` at commit 4228520 (local record; no new fetch performed).
Final deployment URL	[Insert verified Streamlit deployment URL]
Final deployment status	**Pending** redeployment and verification still required.

Chapter 7 Testing

Tests were executed during project and report preparation; the status column reflects the evidence collected. A test is marked Not Run if it cannot be performed. The model and metric tests were run on 11 October 2026 by loading the saved files; training from scratch was run earlier in the project and produced identical metrics. The notebook was also executed from start to finish with no error cells. Test T12 used Streamlit's builtin test runner for a repeatable check, plus a real local server and a browser screenshot

ID	Test description	Expected result	Actual result	Status
T01	Load the Kaggle CSV	File exists; 918 rows, 12 columns	918 × 12 loaded	Pass
T02	Check the four selected columns and target exist	`Age`, `Sex`, `ChestPainType`, `MaxHR`, `HeartDisease` present	All present; validated by `load_kaggle()`	Pass
T03	Validate target values	Only 0 and 1	Counts {'0': 410, '1': 508}	Pass
T04	Feature matrix has exactly four predictors, no target	4 columns, target excluded	['Age', 'Sex', 'ChestPainType', 'MaxHR']	Pass
T05	Training completes and preprocessing is fitted	Pipeline fitted on the training split	Steps ['preprocess', 'model']; fitted: True	Pass
T06	Reload the saved model	Loads; same metrics as inmemory model	Loaded; artifact metrics match recomputed: True	Pass
T07	Prediction shape and labels	184 predictions, labels in {0, 1}	184 predictions, labels [0, 1]	Pass
T08	Reproduce the evaluation metrics	Match `metrics.json` and the notebook	Accuracy 0.788, AUC 0.859; `metrics.json` match: True	Pass
T09	Reproduce the Cleveland baseline	60.0 % accuracy as in original notebook	0.600; saved baseline model 0.600	Pass
T10	Unknown chestpain category (`XYZ`)	No crash (ignored by encoder)	Predicted 1 without error	Pass
T11	Input missing the `MaxHR` column	Error rather than silent output	raised ValueError	Pass
T12	Streamlit app starts without exceptions	No exception	0 exceptions (Streamlit AppTest); also served on localhost and rendered in a browser	Pass
T13	App shows exactly four inputs with correct options	Age, Sex, Chest Pain Type, Max Heart Rate	Number inputs: ['Age (years)', 'Max Heart Rate Achieved']; select boxes: ['Sex', 'Chest Pain Type']	Pass
T14	App generates predictions for valid inputs	Prediction text for each input	60/M/ASY/120 → indicated; 30/M/ATA/170 → not indicated; no exceptions	Pass
T15	App uses the new model file	Loads `heart_disease_model.pkl` with Kaggle features	Local app loads the artifact whose features are Age, Sex, ChestPainType, MaxHR	Pass
T16	Outofrange manual input in the browser	Rejected by widget limits	Not exercised in a browser; limits are set in code	Not Run
T17	Redeploy final version; deployed app runs the latest model	Public URL serves the updated model	Old deployment was asleep (version not verified); no redeployment verified	Not Run

Chapter 8 Limitations and Future Improvements

8.1 Limitations

- **Data:** the model reflects its dataset's quality and mix of populations. Some columns in the data (zero cholesterol) show that the source records are imperfect.
- **Four predictors:** the assignment constraint leaves out variables such as blood pressure, ECG and exercisetest results that would normally matter.
- **Generalisation:** one internal split; no external or temporal validation.
- **Dataset differences:** the Cleveland and Kaggle experiments are not directly comparable, as discussed in Chapter 5.
- **Intended use:** this is an educational prediction system, not a clinical diagnostic tool; it has not been validated against clinical standards.

8.2 Possible Future Improvements

The following are suggestions only; none has been implemented.

- Use repeated or nested crossvalidation with confidence intervals to measure variability.
- Tune SVC hyperparameters with crossvalidation on the training data only.
- Evaluate on an independent dataset and examine performance by sex and age group.
- Calibrate probabilities (for example with `CalibratedClassifierCV`) before showing any risk percentage.
- Add more features after clinical review, if the fourfeature constraint is lifted.
- Automate tests and deployment checks (for example with continuous integration).

Chapter 9 Conclusion

A heartdisease prediction system was built from data to web application. It uses the Kaggle Heart Failure Prediction dataset (918 records) with four inputs Age, `Sex`, `ChestPainType` and `MaxHR` and a scikitlearn pipeline combining scaling, onehot encoding and a Support Vector Classifier. The fitted pipeline is saved as one file and used by a Streamlit application.

On the heldout test set (184 records) the model achieved accuracy 0.788, precision 0.794, recall 0.833, F1score 0.813 and ROCAUC 0.859. The Cleveland baseline reached 0.600 on 60 records; because the experiments differ in several ways, the comparison is informative but not controlled.

The application was tested locally and works as intended. Cloud deployment of the final version was not verified: the earlier deployed app was asleep and its version is unknown, so redeployment remains a pending step.

Main lessons were that preprocessing must be part of the saved model, that categorical codes from different datasets must not be assumed to match, that evaluation must use data not seen in training, and that small test sets and differing datasets limit what comparisons can show.

References

[1] UCI Machine Learning Repository, “heart disease,” dataset page. [Online]. Available: https://archive.ics.uci.edu/dataset/45/heart+disease

[2] fedesoriano, “Heart Failure Prediction Dataset,” Kaggle, Sep. 2021. [Online]. Available: https://www.kaggle.com/datasets/fedesoriano/heartfailureprediction (accessed Oct. 11, 2026). Licence listed on Kaggle: ODbL 1.0.

[3] haqnawaz99, “mlteachingnotebooks (Titanic Passenger Survival Prediction),” GitHub repository. [Online]. Available: https://github.com/haqnawaz99/mlteachingnotebooks. No licence file was found in the local copy of this repository, so no reuse licence is assumed.

[4] scikitlearn developers, “sklearn.svm.SVC.” [Online]. Available: https://scikitlearn.org/stable/modules/generated/sklearn.svm.SVC.html

[5] scikitlearn developers, “sklearn.compose.ColumnTransformer.” [Online]. Available: https://scikitlearn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html

[6] scikitlearn developers, “sklearn.preprocessing.StandardScaler” and “sklearn.preprocessing.OneHotEncoder.” [Online]. Available: https://scikitlearn.org/stable/modules/preprocessing.html

[7] scikitlearn developers, “sklearn.pipeline.Pipeline.” [Online]. Available: https://scikitlearn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html

[8] scikitlearn developers, “Model evaluation: quantifying the quality of predictions.” [Online]. Available: https://scikitlearn.org/stable/modules/model_evaluation.html

[9] Streamlit Inc., “Streamlit documentation” and “Streamlit Community Cloud.” [Online]. Available: https://docs.streamlit.io

[10] F. Pedregosa et al., “Scikitlearn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.